



CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

BÀI 1: Độ phức tạp và tư duy phân tích thuật toán

PHẦN 1



Vì sao phải phân tích độ phức tạp?

- Cùng một bài toán có thể có nhiều thuật toán giải khác nhau, chạy nhanh chậm khác nhau rất nhiều khi dữ liệu lớn.
- Không thể chỉ dựa vào “chạy thử vài input nhỏ” để đánh giá, nên cần một cách đo độc lập với máy tính cụ thể.
- Ý tưởng: đếm số bước tính toán mà thuật toán phải thực hiện khi input có kích thước n , sau đó biểu diễn bằng một hàm $T(n)$.

Khái niệm hàm $T(n)$ và phép toán cơ bản

- Kích thước input thường ký hiệu là n (ví dụ: số phần tử trong mảng, số đỉnh của đồ thị...).
- Ta chọn một loại “phép toán cơ bản” để đếm, ví dụ: phép so sánh, phép gán, phép cộng, hay số lần lặp vòng for.
- $T(n)$ là số phép toán cơ bản thuật toán thực hiện khi input có kích thước n ; mục tiêu là gần đúng $T(n)$ bằng một biểu thức đơn giản theo n .

Đếm đơn giản

```
def add_three(a, b, c):  
    s1 = a + b  
    s2 = s1 + c  
    return s2
```

- Dòng $s1 = a + b$: 1 phép cộng, 1 phép gán $\rightarrow$ coi là 1–2 phép toán (chỉ cần biết là hằng số).
- Dòng $s2 = s1 + c$: tương tự, thêm ~ 1 –2 phép toán.
- `return s2`: 1 thao tác trả về.
- Tổng cộng: $T(n) \approx 3$ –5 phép toán, nhưng không phụ thuộc n , nên độ phức tạp là $O(1)$ – thời gian hằng số.

Đếm trong vòng lặp for

- $total = 0$: 1 phép gán.
- Vòng `for x in arr`: lặp đúng n lần nếu mảng có n phần tử.
- Mỗi lần lặp:
 - Đọc x từ mảng: coi là 1 phép.
 - Tính $total + x$: 1 phép cộng.
 - Gán lại cho $total$: 1 phép gán.
- Mỗi vòng lặp $\sim 2-3$ phép toán, nhân với n vòng $\rightarrow$ khoảng $2n-3n$ phép toán.
- Bỏ các hằng số, ta nói $T(n)$ là $O(n)$ (tuyến tính theo n).

```
def sum_array(arr):  
    total = 0  
    for x in arr:  
        total = total + x  
    return total
```

Đếm trong vòng lặp for (dùng chỉ số)

- $n = \text{len}(\text{arr})$: 1 phép.
- `for i in range(n)`: vòng lặp chạy n lần.
- Mỗi lần lặp:
 - Truy cập `arr[i]`: 1 thao tác.
 - `total = total + arr[i]`: 1 cộng, 1 gán.
- Tổng phép toán vẫn dạng $c_1 + c_2 \cdot n$, nên độ phức tạp vẫn là $O(n)$.

```
def sum_array_index(arr):  
    total = 0  
    n = len(arr)  
    for i in range(n):  
        total = total + arr[i]  
    return total
```

Vòng lặp lồng nhau (nested loop)

- Vòng ngoài `for i in range(n)` chạy n lần.
- Với mỗi i , vòng trong `for j in range(n)` lại chạy n lần.
- Tổng số lần thực thi lệnh `print(arr[i], arr[j])` là $n * n = n^2$.
- Mỗi lần `print` là 1 “bước” (bỏ qua chi tiết), nên $T(n) \approx n^2 \rightarrow$ độ phức tạp $O(n^2)$.

```
def print_pairs(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(n):  
            print(arr[i], arr[j])
```


Vòng lặp while

- Mỗi vòng while:
 - Kiểm tra điều kiện $n > 0$: 1 so sánh.
 - Thực hiện $n = n - 1$: 1 trừ, 1 gán.
- Vòng lặp chạy đúng n lần (mỗi lần giảm 1 cho đến 0).
- Tổng số phép toán $\approx c \cdot n \rightarrow T(n)$ là $O(n)$.

```
def count_down(n):  
    while n > 0:  
        n = n - 1
```

Vòng while với bước nhảy (chia đôi)

```
def divide_by_two(n):  
    count = 0  
    while n > 0:  
        n = n // 2  
        count = count + 1  
    return count
```

- Mỗi vòng: kiểm tra $n > 0$, rồi $n = n // 2$ và tăng count.
- n giảm rất nhanh: $n, n/2, n/4, \dots$ cho đến 1.
- Số lần lặp $\approx$ số lần cần thiết để $\frac{n}{k^k} \leq 1$, tức là $k \approx \log_k n$.
- Vì vậy $T(n)$ là $O(\log n)$, đây là ví dụ điển hình của thuật toán logarit.

Động cơ dùng ký hiệu Big-O

- Khi phân tích $T(n)$, ta không cần chính xác từng hằng số, chỉ cần biết “tốc độ tăng” theo n .
- Ví dụ: $3n + 10$ và $100n + 5$ đều tăng tuyến tính, nên đều được xem là $O(n)$.
- Mục tiêu: phân loại thuật toán thành các “lớp tốc độ”:
 $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$... để tiện so sánh.

Định nghĩa Big-O

- Nói “ $T(n)$ là $O(f(n))$ ” nghĩa là: khi n đủ lớn, $T(n)$ không tăng nhanh hơn một hằng số nhân với $f(n)$.
- Trực quan: Big-O là “cận trên” cho độ lớn của $T(n)$ khi n rất lớn.
- Ví dụ: $T(n) = 3n^2 + 5n + 1 \rightarrow T(n)$ là $O(n^2)$ vì phần n^2 chi phối khi n lớn, các phần còn lại ít quan trọng.

Ví dụ Big-O với hàm nhiều thành phần

- Bước 1: vòng for đơn $\rightarrow O(n)$.
- Bước 2: vòng lặp lồng nhau $\rightarrow O(n^2)$.
- Tổng $T(n) \approx n + n^2 \rightarrow$ phần n^2 lớn hơn rất nhiều khi n lớn, nên độ phức tạp chung là $O(n^2)$.

```
def example(arr):  
    # bước 1: duyệt n phần tử  
    for x in arr:  
        do_something(x)  
    # bước 2: duyệt lồng n x n  
    for i in range(len(arr)):  
        for j in range(len(arr)):  
            do_something_pair(i, j)
```

Ký hiệu Omega (Ω) và ý nghĩa

- Omega (Ω) là “cận dưới”: $T(n)$ là $\Omega(f(n))$ nếu $T(n)$ không thể nhỏ hơn $c \cdot f(n)$ khi n đủ lớn.
- Trực quan: cho biết “ít nhất” thuật toán phải làm bao nhiêu việc.
- Ví dụ: khi cần đọc toàn bộ n phần tử mảng, thời gian luôn $\geq c \cdot n$, nên ta có $T(n)$ là $\Omega(n)$.

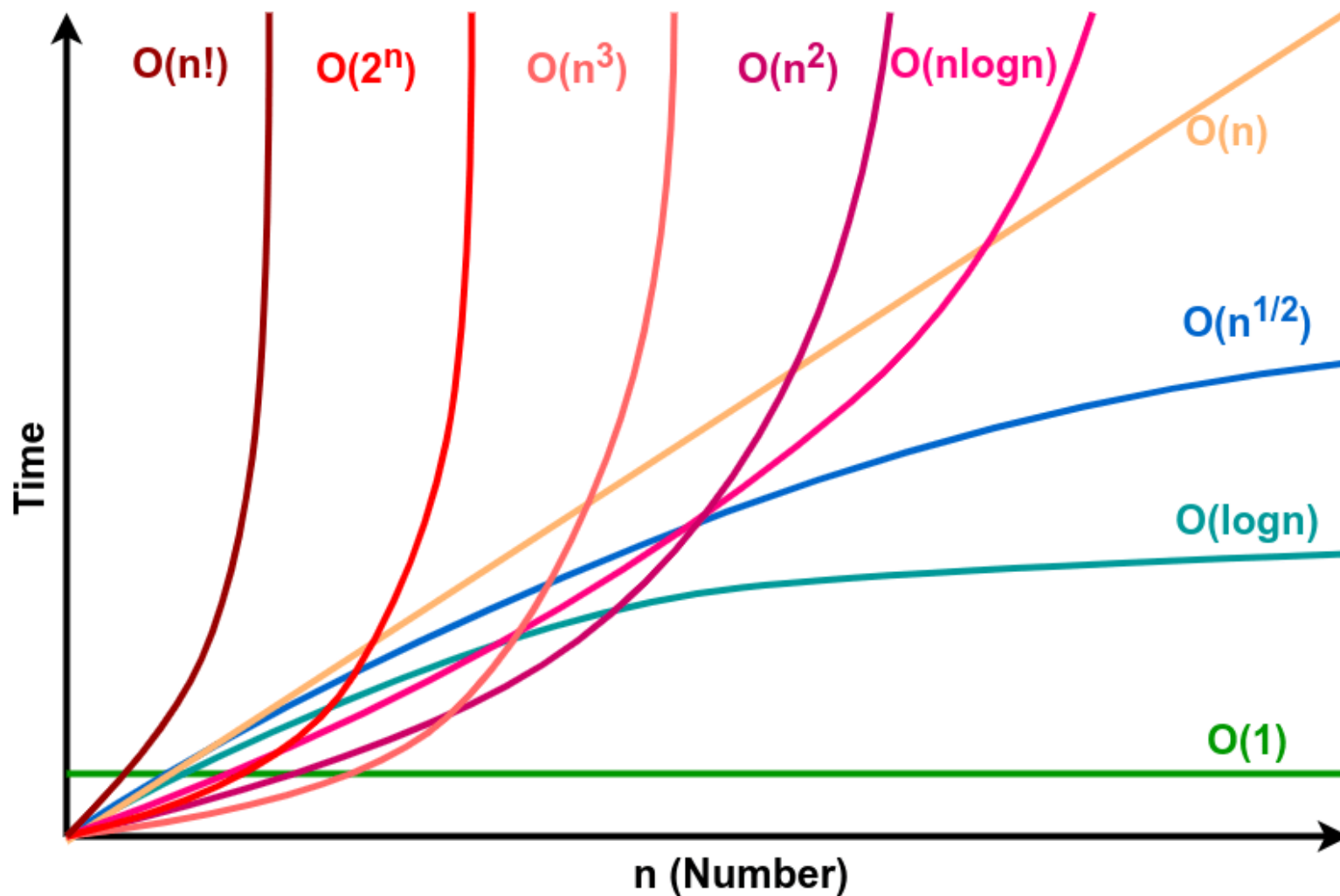
Ký hiệu Theta (Θ) – “đúng bậc”

- Theta (Θ) kết hợp cả Big-O và Omega: $T(n)$ là $\Theta(f(n))$ nếu vừa là $O(f(n))$ vừa là $\Omega(f(n))$.
- Trực quan: $f(n)$ mô tả đúng “bậc tăng” của $T(n)$, không dư, không thiếu.
- Ví dụ: $T(n) = 3n^2 + 5n + 2$ là $\Theta(n^2)$ vì:
 - Không lớn hơn $C1 \cdot n^2$ (Big-O).
 - Cũng không nhỏ hơn $C2 \cdot n^2$ (Omega).

Bảng một số lớp độ phức tạp phổ biến

- $O(1)$: hằng số – truy cập 1 phần tử của mảng cố định, tính vài phép toán đơn lẻ.
- $O(\log n)$: chia đôi dữ liệu mỗi bước, như binary search, thuật toán chia để trị.
- $O(n)$: duyệt tuyến tính một mảng hoặc danh sách.
- $O(n \log n)$: nhiều thuật toán sắp xếp nhanh/merge sort.
- $O(n^2)$: các thuật toán dùng 2 vòng lặp lồng nhau trên n phần tử.

Đồ thị biểu diễn một số độ phức tạp phổ biến



Khái niệm Best/average/worst case

- Best case: trường hợp dữ liệu thuận lợi nhất, thuật toán chạy nhanh nhất.
- Worst case: dữ liệu xấu nhất, thuật toán chạy lâu nhất (thường được dùng để đảm bảo giới hạn).
- Average case: thời gian trung bình nếu dữ liệu ngẫu nhiên, khó tính toán hơn, thường cần xác suất.

Ví dụ search tuyến tính

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

- Thuật toán duyệt tuần tự từ đầu đến cuối mảng, dừng khi thấy target hoặc hết mảng.
- Số bước phụ thuộc vào vị trí của target trong mảng (hoặc có/không tồn tại).

Phân tích linear search: best case

- Best case: target nằm ở vị trí đầu tiên `arr[0]`.
- Khi đó:
 - Vòng for chạy 1 lần.
 - Thực hiện 1 lần so sánh `arr[0] == target`, rồi return.
- $T(n)$ trong best case là hằng số, viết là $O(1)$, còn cận dưới là $\Omega(1)$ (chỉ cần 1 lần so sánh).

Phân tích linear search: worst case và average

Worst case:

- target không tồn tại trong mảng, hoặc nằm ở phần tử cuối cùng.
- Vòng for chạy đủ n lần, mỗi lần 1 so sánh.
- $T(n) \approx n \rightarrow O(n)$.

Average case:

- Nếu giả sử target xuất hiện ngẫu nhiên ở một vị trí, trung bình phải kiểm tra khoảng $n/2$ phần tử.
- $T(n)$ trung bình $\approx n/2 \rightarrow$ vẫn là $O(n)$.

PHẦN 2



Độ phức tạp “ẩn” trong Python

- Không phải mọi thao tác đều là $O(1)$!
- Một số thao tác tưởng “đơn giản” nhưng thực ra tốn $O(n)$

```
if x in my_list: #  $O(n)$  - phải duyệt cả list
    do_something()

count = my_list.count(5) #  $O(n)$  - đếm qua từng phần tử

my_list.remove(x) #  $O(n)$  - tìm và xóa phải dịch chuyển
```

- Những thao tác này khi nằm trong vòng lặp sẽ làm tăng độ phức tạp!

Ví dụ 1: Kiểm tra trùng lặp (cách chậm)

Bài toán: Kiểm tra xem list có phần tử trùng lặp không?

- Vòng ngoài: i chạy từ 0 đến n-1 (n lần)

- Vòng trong: j chạy từ i+1

đến n-1 (giảm dần từ n-1 đến 0 lần)

- Tổng số so sánh: $(n-1) + (n-2)$

$$+ \dots + 1 = n(n-1)/2 \approx n^2/2$$

- Độ phức tạp: $O(n^2)$

```
def has_duplicates_v1(arr):  
    """Cách 1: So sánh từng cặp"""  
    n = len(arr)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if arr[i] == arr[j]:  
                return True  
    return False
```


Ví dụ 1: Kiểm tra trùng lặp (cách nhanh)

- Vòng for chạy n lần
- Mỗi lần:
 - item in seen: $O(1)$ trung bình với set
 - seen.add(item): $O(1)$
- Tổng: $n \times (1 + 1) = 2n$
- Độ phức tạp: $O(n)$

```
def has_duplicates_v2(arr):  
    """Cách 2: Dùng set"""  
    seen = set()  
    for item in arr:  
        if item in seen: #  $O(1)$  với set  
            return True  
        seen.add(item) #  $O(1)$   
    return False
```

Nhanh hơn 500 lần với $n = 1000$!

So sánh thực tế hai cách

Với $n = 10,000$:

Cách 2 nhanh

hơn ~2500 lần!

```
import time

# Test với n = 10,000
arr = list(range(10000))
arr.append(5000) # Thêm 1 phần tử trùng

# Cách 1:  $O(n^2)$ 
start = time.time()
result1 = has_duplicates_v1(arr)
time1 = time.time() - start
print(f"Cách 1: {time1:.4f} giây") # ~2.5 giây

# Cách 2:  $O(n)$ 
start = time.time()
result2 = has_duplicates_v2(arr)
time2 = time.time() - start
print(f"Cách 2: {time2:.4f} giây") # ~0.001 giây
```

Ví dụ 2: Tìm cặp có tổng bằng target (cách chậm)

Bài toán: Cho mảng số nguyên và số target, tìm 2 số có tổng = target

- Vòng ngoài: n lần
- Vòng trong: $(n-1) + (n-2) + \dots + 1$ lần
- Mỗi lần:

1 phép cộng + 1 so sánh = 2 phép toán

- Tổng: $n(n-1)/2 \times 2 \approx n^2$
- Độ phức tạp: $O(n^2)$

```
def two_sum_v1(arr, target):  
    """Cách 1: Brute force - thử mọi cặp"""  
    n = len(arr)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if arr[i] + arr[j] == target:  
                return (i, j)  
    return None
```

Ví dụ 2: Tìm cặp có tổng bằng target (cách nhanh)

- Vòng for: n lần
- Mỗi lần:
 - Tính complement: $O(1)$
 - Kiểm tra complement in seen: $O(1)$ với dict
 - Gán `seen[arr[i]] = i`: $O(1)$
- Tổng: $n \times 3 = 3n$
- Độ phức tạp: $O(n)$

```
def two_sum_v2(arr, target):  
    """Cách 2: Dùng dict lưu trữ"""  
    seen = {} # key: giá trị đã thấy, value: chỉ số  
  
    for i in range(len(arr)):  
        complement = target - arr[i]  
  
        if complement in seen: #  $O(1)$   
            return (seen[complement], i)  
  
        seen[arr[i]] = i # Lưu giá trị và chỉ số  
  
    return None
```

Giải thích tại sao dict/set nhanh

Hash table (cơ chế đằng sau dict và set):

So với list:

- if key in my_list:

Phải duyệt tuần tự $\rightarrow O(n)$

- if key in my_set:

Tính hash và tra $\rightarrow O(1)$

```
# Khi ta làm: seen[key] = value
# Python thực hiện:
hash_value = hash(key) # Tính hash O(1)
index = hash_value % table_size # Tìm vị trí O(1)
table[index] = value # Ghi vào O(1)

# Khi ta làm: if key in seen
# Python thực hiện tương tự:
hash_value = hash(key)
index = hash_value % table_size
return table[index] is not None # O(1)
```

List slicing và độ phức tạp

Slicing tạo bản sao mới $\rightarrow O(k)$ với k là độ dài slice

- Vòng for: n lần
- Lần 1: slice `arr[0:]` có độ dài $n \rightarrow O(n)$
- ...
- Lần n : slice `arr[n-1:]` có độ dài 1 $\rightarrow O(1)$
- Tổng: $n + (n-1) + \dots + 1 = n(n+1)/2 \approx n^2/2$
- Độ phức tạp: $O(n^2)$ chỉ riêng phần slicing!

```
def process_data(arr):  
    n = len(arr)  
    for i in range(n):  
        sub_arr = arr[i:] # Slicing  $O(n-i)$   
        do_something(sub_arr)
```

Tối ưu slicing bằng index

Lợi ích

- Không tạo bản sao → Tiết kiệm bộ nhớ
- Truy cập trực tiếp `arr[j]` → $O(1)$ thay vì $O(k)$
- Độ phức tạp tổng thể giảm từ $O(n^2)$ xuống $O(n)$

```
def process_data_optimized(arr):  
    """Thay vì slicing, dùng index"""  
    n = len(arr)  
    for i in range(n):  
        # Không tạo sub_arr, chỉ truyền index  
        do_something_with_index(arr, i)  
  
def do_something_with_index(arr, start):  
    """Xử lý từ index start trở đi"""  
    for j in range(start, len(arr)):  
        process(arr[j]) # Truy cập trực tiếp  $O(1)$ 
```

Ví dụ 3: Đếm tần suất (cách chậm)

Bài toán: Đếm số lần xuất hiện của mỗi phần tử

- Vòng ngoài: n lần
- Mỗi lần:
 - `item not in processed`: Trung bình $O(n/2) = O(n)$
 - `arr.count(item)`: $O(n)$
- Tổng: $n \times (n + n) = 2n^2$
- Độ phức tạp: $O(n^2)$

```
def count_frequency_v1(arr):  
    """Cách 1: Dùng list.count()"""  
    result = []  
    processed = []  
  
    for item in arr: # O(n)  
        if item not in processed: # O(p) với p = len(processed)  
            count = arr.count(item) # O(n)  
            result.append((item, count))  
            processed.append(item)  
  
    return result
```


Ví dụ 3: Đếm tần suất (cách nhanh)

```
from collections import defaultdict

def count_frequency_v3(arr):
    freq = defaultdict(int)
    for item in arr:
        freq[item] += 1 # Tự động khởi tạo 0 nếu chưa có
    return list(freq.items())
```

Độ phức tạp: $O(n)$ - Nhanh hơn rất nhiều!

Ví dụ 4: Tìm max (so sánh các cách)

```
# Cách 1: Built-in (tối ưu nhất)
def find_max_v1(arr):
    return max(arr) # O(n), viết bằng C
```

```
# Cách 3: Duyệt 1 lần (tối ưu)
def find_max_v3(arr):
    if not arr:
        return None
    max_val = arr[0]
    for i in range(1, len(arr)): # Không slicing
        if arr[i] > max_val:
            max_val = arr[i]
    return max_val # O(n), không tốn bộ nhớ thêm
```

```
# Cách 2: Duyệt 1 lần (chuẩn)
def find_max_v2(arr):
    if not arr:
        return None
    max_val = arr[0]
    for item in arr[1:]: # ⚠ Slicing O(n)
        if item > max_val:
            max_val = item
    return max_val # Tổng O(n) + O(n) = O(n), nhưng tốn bộ nhớ
```

Kết luận: Cả 3 đều $O(n)$, nhưng v1 nhanh nhất (C code), v3 tiết kiệm bộ nhớ hơn v2.

Nested loop với điều kiện phức tạp

Phân tích chi tiết:

- $i = 0$: vòng j chạy n lần
- ...
- $i = n-1$: vòng j chạy 1 lần
- Tổng: $n + (n-1) + \dots + 1 = n(n+1)/2$
- Độ phức tạp: $O(n^2)$

```
def complex_nested(arr):  
    n = len(arr)  
    count = 0  
  
    for i in range(n):          # n lần  
        for j in range(i, n):  # (n-i) lần  
            if arr[i] + arr[j] > 10: # O(1)  
                count += 1  
  
    return count
```

Lưu ý: Dù `range(i, n)` giảm số lần lặp, vẫn là $O(n^2)$ vì tổng vẫn tỷ lệ với n^2 .

So sánh các thao tác list thường dùng

Thao tác	Độ phức tạp	Giải thích
<code>arr[i]</code>	$O(1)$	Truy cập trực tiếp theo index
<code>arr.append(x)</code>	$O(1)$ amortized	Thêm vào cuối
<code>arr.insert(0, x)</code>	$O(n)$	Phải dịch chuyển tất cả phần tử
<code>arr.pop()</code>	$O(1)$	Xóa phần tử cuối
<code>arr.pop(0)</code>	$O(n)$	Phải dịch chuyển tất cả phần tử
<code>x in arr</code>	$O(n)$	Duyệt tìm kiếm tuần tự
<code>arr.remove(x)</code>	$O(n)$	Tìm + xóa + dịch chuyển
<code>arr.sort()</code>	$O(n \log n)$	Timsort algorithm
<code>arr[i:j]</code>	$O(j-i)$	Tạo bản sao

Ví dụ 5: Loại bỏ phần tử trùng lặp

```
def remove_duplicates_v1(arr):  
    """Cách 1: Dùng list và in/remove"""  
    result = []  
    for item in arr: # O(n)  
        if item not in result: # O(len(result)) ≈ O(n)  
            result.append(item) # O(1)  
    return result  
# Độ phức tạp: O(n²)
```

Kết luận: v3 là tối ưu nhất ($O(n)$
+ giữ thứ tự).

```
def remove_duplicates_v2(arr):  
    """Cách 2: Dùng set rồi chuyển về list"""  
    return list(set(arr)) # O(n) + O(n) = O(n)  
# Lưu ý: Mất thứ tự ban đầu
```

```
def remove_duplicates_v3(arr):  
    """Cách 3: Dùng set nhưng giữ thứ tự"""  
    seen = set()  
    result = []  
    for item in arr: # O(n)  
        if item not in seen: # O(1)  
            seen.add(item) # O(1)  
            result.append(item) # O(1)  
    return result # O(n), giữ thứ tự
```

Ví dụ 6: Intersection của 2 list

```
def intersection_v1(arr1, arr2):
    """Cách 1: Dùng in với list"""
    result = []
    for item in arr1: # O(n)
        if item in arr2 and item not in result: # O(m) + O(k)
            result.append(item)
    return result
# Độ phức tạp: O(n × (m + k)) ≈ O(n × m)
```

Với $n = m = 10000$:

v1 mất ~5s,

v2/v3 mất ~0.002s!

```
def intersection_v3(arr1, arr2):
    """Cách 3: Dùng set intersection"""
    return list(set(arr1) & set(arr2)) # O(n + m)
```

```
def intersection_v2(arr1, arr2):
    """Cách 2: Chuyển 1 list thành set"""
    set2 = set(arr2) # O(m)
    result = []
    seen = set()
    for item in arr1: # O(n)
        if item in set2 and item not in seen: # O(1) + O(1)
            result.append(item)
            seen.add(item)
    return result
# Độ phức tạp: O(m + n)
```

Bài tập thực hành

Phân tích độ phức tạp các đoạn code sau:

```
def mystery1(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(n):  
            if i != j and arr[i] == arr[j]:  
                return True  
    return False
```

```
def mystery2(arr):  
    seen = {}  
    for item in arr:  
        if item in seen:  
            return True  
        seen[item] = True  
    return False
```

```
def mystery3(arr):  
    result = []  
    for i in range(len(arr)):  
        temp = arr[:i] # Slicing  
        result.append(sum(temp)) # sum() là O(i)  
    return result
```

Tóm tắt

- Không phải mọi thao tác Python đều $O(1)$: in, count, remove, insert(0) đều là $O(n)$
- Set/dict là công cụ tối ưu mạnh mẽ: Chuyển từ $O(n^2) \rightarrow O(n)$ trong nhiều bài toán
- Slicing tạo bản sao: Tránh dùng trong vòng lặp, thay bằng index
- Nested loops: Luôn tính tổng số lần lặp thực tế, không chỉ nhìn số vòng



Kết thúc